22.4 工具与 Agent 的协作

> **生成模型**：Claude Opus 4.6 (anthropic/claude-opus-4-6)
> **Token 消耗**：输入 ~900k tokens，输出 ~85k tokens（本章合计）

工具不是孤立运行的——它们与 Agent 的提示系统、对话上下文、多轮交互深度协作。本节分析工具如何融入 Agent 的推理循环。

22.4.1 工具提示注入（tool description → system prompt）

工具描述的双重身份

每个工具的 `description` 字段既是 JSON Schema 的一部分（发送给 LLM 的工具定义），也是系统提示词的信息源。OpenClaw 通过 `buildToolSummaryMap()` 将工具描述汇总为一个映射表：

```
// src/agents/tool-summaries.ts
export function buildToolSummaryMap(tools: AgentTool[]): Record<string, string> {
  const summaries: Record<string, string> = {};
  for (const tool of tools) {
    const summary = tool.description?.trim() || tool.label?.trim();
    if (summary) {
      summaries[tool.name.toLowerCase()] = summary;
    }
  }
  return summaries;
}
```

通道特定的工具提示

不同通道会为同一个工具提供额外的使用提示。例如 MS Teams 通道会告诉 Agent 如何用 `message` 工具发送 Adaptive Cards：

```
// extensions/msteams/src/channel.ts
agentPrompt: {
  messageToolHints: () => [
    "- Adaptive Cards supported. Use action=send with card={type,version,body}.",
    "- MSTeams targeting: omit target to reply to current conversation.",
  ],
},
```

这些提示会被注入到系统提示词中，帮助 Agent 理解在当前通道中如何正确使用工具。

工具可用性影响系统提示

工具策略过滤后的工具集直接影响 Agent 的行为。如果 `web_search` 被禁用，Agent 的系统提示中不会包含搜索相关的指引，也不会尝试调用该工具。这种"工具可用性驱动提示"的机制确保了 Agent 不会尝试调用不存在的工具。

系统提示报告（`system-prompt-report.ts`）会记录当前可用的工具列表：

```
// src/agents/system-prompt-report.ts
function buildToolsEntries(tools: AgentTool[]) {
  return tools.map((tool) => ({
    name: tool.name,
    description: tool.description?.slice(0, 200),
  }));
}
```

22.4.2 多轮工具调用模式

单轮 vs 多轮

LLM 的工具调用有两种模式：

单轮调用：LLM 在一次回复中调用一个或多个工具，等待结果后生成最终回复。

```
用户: "今天天气怎么样？"
Agent: → tool_use: web_search("今天天气")
       ← tool_result: "北京 晴 25°C"
       → "今天北京天气晴朗，气温 25°C。"
```

多轮调用：LLM 需要多次工具调用才能完成任务，每次调用的结果用于指导下一次调用。

```
用户: "帮我在飞书创建一个项目文档，包含最近的会议记录"
Agent: → tool_use: sessions_history(limit=5)
       ← tool_result: [{ ... 会议记录 ... }]
       → tool_use: feishu_doc_create(title="项目文档", content=...)
       ← tool_result: { docId: "abc123", url: "..." }
       → tool_use: message(action="send", message="文档已创建: ...")
       ← tool_result: { ok: true }
       → "已创建项目文档并发送了链接。"
```

工具调用上下文的维护

在多轮调用中，每次工具结果都会被追加到对话历史中，成为后续推理的上下文：

```
messages: [
  { role: "user", content: "帮我查一下项目进度" },
  { role: "assistant", tool_calls: [{ name: "sessions_list", params: {} }] },
  { role: "tool", content: "[{ session_id: ... }]" },
  { role: "assistant", tool_calls: [{ name: "sessions_history", params: { id: "..." } }
  { role: "tool", content: "[{ message: ... }]" },
  { role: "assistant", content: "根据最近的会话记录，项目进度..." },
]
```

这意味着**工具结果占用对话上下文窗口**。如果工具返回大量数据（如完整的文件内容或长列表），会快速消耗上下文容量。

> **衍生解释**：**上下文窗口**（Context Window）是 LLM 能同时处理的文本长度限制，通常以 Token 计量。Claude 支持约 200k tokens 的上下文窗口，GPT-4 支持 128k tokens。工具调用的每个参数、每个返回结果都计入上下文消耗。因此，工具设计需要平衡信息量和上下文效率。

并行工具调用

现代 LLM 支持在单次回复中发起**多个并行**的工具调用：

```
Agent: → tool_use: [
  { name: "web_search", params: { query: "公司A 财报" } },
  { name: "web_search", params: { query: "公司B 财报" } },
]
```

OpenClaw 的 pi-agent-core 支持并行执行这些调用，并将所有结果一起返回。

22.4.3 工具结果对会话上下文的影响

上下文压力

工具结果是上下文消耗的主要来源之一。一些典型的工具结果大小：

工具	典型结果大小	上下文影响
`web_search`	1-5k tokens	中等
`web_fetch`	5-50k tokens	高（可能触发压缩）
`read` （大文件）	10-100k tokens	非常高
`exec` （长输出）	1-50k tokens	高
`sessions_list`	0.5-2k tokens	低
`browser` （截图）	图片不计入文本 token	特殊

tool_result_persist 的上下文优化

`tool_result_persist` 钩子的核心价值之一就是**优化上下文使用**。插件可以在工具结果持久化之前精简其内容：

```
// 一个假设的插件实现
api.on("tool_result_persist", (event, ctx) => {
  if (ctx.toolName === "exec" && event.message.content?.length > 10000) {
    // 截断过长的命令输出
    return {
      message: {
        ...event.message,
        content: event.message.content.slice(0, 5000) + "\n[... 输出已截断 ...]",
      },
    };
  }
});
```

与上下文压缩的交互

当工具调用导致上下文接近窗口限制时，会触发第 9 章介绍的上下文压缩机制（Compaction）。压缩过程中，旧的工具调用及其结果会被 LLM 总结为更简短的摘要，释放上下文空间。

这形成了一个循环：

```
工具调用 → 上下文增长 → 触发压缩 → 释放空间 → 继续工具调用
```

压缩前后， `before_compaction` 和 `after_compaction` 钩子会被触发，插件有机会在压缩前保存关键数据或在压缩后恢复状态。

工具与 Agent 的反馈循环

实际运行中，工具系统与 Agent 形成了一个**反馈循环**：

1. **Agent 规划**：基于用户请求和当前上下文，Agent 决定需要调用哪些工具
2. **工具执行**：工具调用产生结果，结果被追加到上下文
3. **Agent 评估**：Agent 根据工具结果评估是否需要进一步操作
4. **迭代或完成**：如果任务未完成，回到步骤 1；否则生成最终回复

这个循环被 pi-agent-core 的运行时管理，直到以下条件之一满足：

- Agent 生成了不包含 tool_use 的回复（认为任务完成）
- 达到最大工具调用轮数限制
- 会话被中断（AbortSignal）
- 上下文窗口用尽且压缩后仍无法继续

本节小结

1. **工具描述同时服务于 LLM 工具定义和系统提示词**，通道特定的工具提示会被注入系统提示，指导 Agent 在特定通道中正确使用工具。
2. **多轮工具调用是 Agent 完成复杂任务的关键模式**，每次工具结果都成为后续推理的上下文。现代 LLM 还支持单次回复中的并行工具调用。
3. **工具结果是上下文消耗的主要来源**， `tool_result_persist` 钩子和上下文压缩机制共同管理上下文压力。
4. **工具与 Agent 形成规划-执行-评估的反馈循环**，由 pi-agent-core 运行时管理，直到任务完成、达到限制或被中断。